

Patron de comportamiento Visitor

Universidad Católica de oriente
Ingeniería en sistemas
Programación Orientada a objetos

Sofia Tobón Atehortúa
Veronica Loaiza Gómez
Juan Ricardo Cogollo
24 de septiembre – 2026

➔ Patrones de diseño-

Un patrón de diseño es una forma conocida de solución a problemas que ocurren con frecuencia en el mundo de la programación.

Existen tres categorías, creación, estructurales y de comportamiento.

Los de comportamiento son: Patrones que de manera organizada hacen que los objetos se “comuniquen” y sepan que acción hacer dependiendo la operación solicitada.

➔ Patron de diseño visitor-

Visitor es un patrón de comportamiento que permite hacer que una clase pueda recibir diferentes operaciones sin tener que modificar la clase cada vez que se quiera añadir una nueva, además también sabemos sobre este patrón que si no se utiliza para códigos grandes se pueden hacer muchas clases para hacer un código y esto hace que el código crezca y que se pueda volver hasta confuso.

Visitor se utiliza cuando existen objetos relacionados entre sí, algunos ejemplos son:

- Comida, bebidas, postres -> hacen parte de una misma categoría la cual llamaremos restaurante
- Ropa, electrodomésticos -> aunque no se parezca que exista una relación entre ellas las podemos colocar en una misma categoría que podríamos llamar almacén.

Ya antes mencionado, pero no menos importante Visitor se puede utilizar cuando se quieren realizar muchas operaciones relacionadas y se quiere en el futuro añadir mas

➔ Concepto de doble despacho-

Explicado de manera sencilla, es como si se hicieran dos preguntas para saber qué decisión tomar, con este contexto doble despacho permite que Visitor sepa que operación ejecutar dependiendo del objeto y el visitante, se le llama doble porque depende de dos tipos de objetos.

Este concepto se utiliza cuando aparte de saber que objetos tenemos también se necesita saber con que otros objetos se está interactuando.

En el código: La instrucción `visitor.Visitar(this);` permite que el Visitor identifique el tipo concreto del producto y ejecute el método `Visitar()` correspondiente. Primero se llama a `Aceptar()` del producto y luego a `Visitar()` según el tipo de producto recibido.

➔ Implementación-

```
5 referencias
interface IVisitor
|
2 referencias
void Visitar(Ropa ropa);
2 referencias
void Visitar(Electronico electronico);
2 referencias
void Visitar(Alimento alimento);
|
```

Define las operaciones que el Visitor puede realizar sobre cada tipo de producto. Cada método **Visitar()** corresponde a un elemento concreto.

```
3 referencias
interface IProducto
{
    6 referencias
    void Aceptar(IVisitor visitor);
}
}
```

Define el método **Aceptar()**, que permite que los productos reciban un Visitor y le permitan realizar una operación sobre ellos.

```
2 referencias
public void Aceptar(IVisitor visitor)
{
    visitor.Visitar(this);
}
}
```

El producto recibe al Visitor y redirige la operación al método **Visitar()** correspondiente a su tipo.

```
2 referencias
class CalculadorImpuesto : IVisitor
{
    2 referencias
    public void Visitar(Ropa ropa)
    {
        Console.WriteLine("Impuesto de ropa: " + ropa.Precio * 0.19);
    }

    2 referencias
    public void Visitar(Electronico electronico)
    {
        Console.WriteLine("Impuesto de electrónico: " + electronico.Precio * 0.19);
    }

    2 referencias
    public void Visitar(Alimento alimento)
    {
        Console.WriteLine("Impuesto de alimento: " + alimento.Precio * 0.05);
    }
}
```

CalculadorImpuesto hereda de **IVisitor** entonces **CalculadorImpuesto** es el Visitor concreto. Contiene la operación que se quiere realizar sobre cada producto.

```
CalculadorImpuesto impuesto = new CalculadorImpuesto();  
  
camisa.Aceptar(impuesto);  
celular.Aceptar(impuesto);  
comida.Aceptar(impuesto);
```

Se crea el Visitor y lo envía a cada producto mediante **Aceptar()**. De esta forma, cada producto ejecuta la operación que le corresponde.

➔ Conclusión:

El patron de comportamiento visitor es muy útil cuando se quieren implementar varias operaciones en varios objetos que están relaciones entre sí. La principal ventaja es que se pueden añadir más operaciones sin modificar las clases originales, además de utilizar el concepto de doble despacho para tomar la decisión de que operación tomar dependiendo del objeto al que se le esta visitando.